

BD 아키텍처 — 코드가 정의하는 정보 흐름 (특히: 이전 블록 → 현재 블록)

기준 코드: `models_seq/bd_models.py` (라인 번호는 현재 저장소 기준) · mask 커널 중심 · 동반 문서: `BD_CODE_VERIFICATION`, `RESULTS_BD`

핵심 답부터. 이전 블록의 정보가 현재 블록의 디노이징에 들어가는 경로는 **self-attention** 단 하나다. (1) **샘플링 시:** 캔버스가 [커밋된 블록들 || 현재 MASK 블록]으로 자라고, block-causal 마스크 하에서 현재 블록의 모든 토큰이 **커밋된 토큰들을 매 레이어에서 직접 attend**한다 — 커밋 토큰은 값이 동결(frozen)되지만 KV 캐시가 없어 매 **forward**마다 재인코딩된다. (2) **학습 시:** $[x_t \parallel x_0]$ 이어붙인 입력에서, 노이즈 블록의 쿼리가 **이전 블록들의 클린 x_0 키/밸류**를 attend한다 (offset-block-causal, 일종의 teacher forcing — 샘플링 시 "커밋 = 클린"과 정확히 대응). adaLN conditioning 경로(시간 t + OD 임베딩)는 이전 블록의 **내용**은 나르지 않는다.

1. 학습 시 정보 흐름 — $[x_t \parallel x_0]$ 와 3중 어텐션 마스크

입력 캔버스 (블록 3개 예시, `_forward_mask` `bd_models.py:416`)



블록마다 독립적인 `t` (`_sample_t_blocks` → `repeat_interleave`, :426-427). 두 절반은 위치 임베딩을 공유한다: `pos = arange(S) % n` (`BDTransformer.forward`, :264) — x_0 절반이 "같은 자리의 클린 버전"으로 인식되는 이유.

어텐션 마스크 (`get_block_diff_mask`, :61-84) — 행 = 쿼리, 열 = 키:

	B1	B2	B3	B1'	B2'	B3'
B1	BD	.	.	.	.	.
B2	.	BD	.	OBC	.	.
B3	.	.	BD	OBC	OBC	.
B1'	.	.	.	BD·BC	.	.
B2'	.	.	.	BC	BD·BC	.
B3'	.	.	.	BC	BC	BD·BC

BD **M_{BD}** (block_diagonal): 같은 절반-같은 블록 내부 self-attention — 블록 내부 일관성 담당 OBC **M_{OBC}** (offset_block_causal): 노이즈 블록 쿼리 → 엄격히 이전 블록의 클린 x_0 키 — "이전 블록 정보"의 학습 시 통로 BC **M_{BC}** (block_causal): 클린 절반끼리의 블록-인과 attention — 뒤 블록이 참조할 클린 표현을 만들어 줌

```
bd_models.py:78-81
block_diagonal      = (bq == bk) & (x0_q == x0_kv)
offset_block_causal = (bq > bk) & x0_kv & ~x0_q    # ← 이전 블록 정보의 통로
block_causal        = (bq >= bk) & x0_kv & x0_q
mask = block_diagonal | offset_block_causal | block_causal
```

그림 1. 학습 마스크. B3(노이즈)의 행을 보면: 자기 블록(BD)과 **이전 블록들의 클린 버전(OBC)**만 보인다 — 이전 블록의 노이즈 버전(B1, B2)은 절대 보지 않는다. 이는 샘플링 시 "이전 블록 = 이미 커밋된 클린 토큰"과 정확히 일치하는 조건화다 (teacher forcing). 마지막에 `logits[:, :n]` 만 취해 노이즈 절반의 예측만 loss에 쓴다 (:447).

2. 샘플링 시 정보 흐름 — 블록 j를 디노이징할 때

캔버스 성장과 고정 프리픽스 (`plan`, :616-624)



블록(B=4) 두 개가 커밋된 상태에서 세 번째 블록이 `_init_block` (:672, 전부 MASK)으로 append된 직후. 노란색 = 클린 prefix 토큰 (dst, ori — 매 블록 append마다 재고정).

디노이징 루프에서의 attention (`_denoise_block_mask`, :689-718)

	B1	B2	B3(cur)
B1	✓	.	.
B2	✓	✓	.
B3(cur)	✓	✓	✓

```

bd_models.py:87-96 (get_block_causal_mask)
blk = arange(s) // block_size
mask = blk[:, None] >= blk[None, :] # 자기 블록 + 모든 이전 블록

bd_models.py:705 (매 reveal마다 전체 시퀀스 forward)
logits = self._cfg_logits(seq, self._t_input(b, s, t*100.0), origs, dests, attn, w)
# seq = [커밋 블록들 || 현재 블록]: 커밋 토큰이 키/밸류로 직접 참여
bd_models.py:677-686 (_t_input): 커밋 블록 t=0, 현재 블록만 t (mask 커널은 time_cond=False라 t 자체도 미사용)

```

- 이전 블록 반영 방식: 6개 레이어 전부에서 현재 블록의 쿼리가 커밋 토큰의 키/밸류를 attend — 별도의 "요약 벡터"나 조건 채널 없이 **토큰 수준 전면 참조**.
- 동결이지만 캐시는 아님: 커밋 토큰의 값은 다시 샘플되지 않지만(revealed ⇒ frozen), KV 캐시가 없어 매 reveal forward마다 재인코딩된다 (bd3lms의 KV 캐시는 torch 1.12 호환을 위해 제거 — 모듈 docstring). 따라서 커밋 토큰의 표현은 현재 블록의 마스킹 상태에 따라 미세하게 달라질 수 있으나, block-causal 마스크상 커밋 토큰의 쿼리는 현재 블록을 보지 못하므로 커밋 표현은 사실상 결정적이다.
- first-hitting: 한 forward당 masked 위치 1개만 공개 (:711-718) — 공개된 토큰은 다음 forward부터 "이전 정보"의 일부가 되어 같은 블록 내 남은 위치들의 문맥이 된다 (블록 내부도 순차 조건화).

그림 2. 샘플링 마스크(블록 단위 표시). 학습의 OBC(노이즈→클린)가 샘플링에서는 "현재 블록→커밋 블록" 참조로 실현된다 — 학습·샘플링의 조건화 분포가 일치하는 것이 \$1 마스크 설계의 목적.

3. 백본 스택 — 토큰이 지나가는 길

tokens (b, S) — 학습: $S=2n$ / 샘플링: $S=s$
x_embedding: node2vec 초기화, V+3 (BDTransformer._init_: :182-195)

↓ in_proj(100→256) + pos_emb[arange(S) % n] (:266)

BDBlock × 6 (:106-154) — 블록당:

- ① adaLN(c) → 6 modulation (shift/scale/gate × attn/MLP, zero-init :131-133)
- ② $x \leftarrow \text{gate} \cdot \text{Attn}(\text{modulate}(\text{LN}(x)), \text{attn_mask})$ — 수동 matmul attention, masked_fill(~mask, -1e9) (:142-143)
- ③ $x \leftarrow \text{gate} \cdot \text{MLP}(\text{modulate}(\text{LN}(x)))$

↓ final LN + adaLN(shift,scale) → zero-init head (:273-274)

logits (b, S, V+3) → 학습: [:, :n] 만 loss / 샘플링: [:, -block:] 만 사용

conditioning 벡터 c (토큰별, :267-268) = time_mlp(t_tok) + od_mlp([emb(ori); emb(dst)]) — adaLN을 통해서만 주입. 즉 c가 나르는 것은 (블록별 노이즈 레벨, OD 조건)뿐이고, 이전 블록의 내용은 c에 없다 — 내용은 전부 attention의 키/밸류로 흐른다. 조건 dropout 시 null_cond로 대체 (:246-248).

그림 3. BDTransformer. bd3lms의 DDIT와 구조 동일 계열이나 (a) rotary 대신 학습형 절대 위치(두 절반 공유), (b) c가 시퀀스당 1개가 아니라 토큰당(블록별 t 지원 + OD 채널), (c) FlexAttention 대신 수동 matmul (torch 1.12).

4. 주목해서 볼 코드 지점 요약

위치 (bd_models.py)	무엇을 결정하나
:78-81 get_block_diff_mask	이전 블록 정보의 학습 시 통로 — 특히 offset_block_causal 한 줄. 이 줄이 없으면 블록들이 서로 독립인 모델이 된다
:87-96 get_block_causal_mask	샘플링 시 현재 블록 → 커밋 블록 참조
:264 pos = arange(S) % n	x_t/x_0 절반의 위치 공유 — OBC가 "같은 자리의 클린 과거"를 가리키게 만드는 장치
:440-447 _forward_mask	$[x_t x_0]$ concat 한 번의 forward로 전 블록 학습(벡터화): logits[:, :n]
:616-624 plan 루프	블록 append + dst/ori 핀 재고정 — 캔버스가 자라며 커밋 정보 축적
:705 _denoise_block_mask	매 reveal마다 전체 시퀀스 forward — 커밋 토큰 재인코딩 (KV 캐시 없음)
:677-686 _t_input	커밋 블록 t=0 / 현재 블록만 t — "클린 vs 진행 중" 구분 (mask 커널은 MASK 토큰 자체가 이를 자기 식별)
:149-153 BDBlock.forward	adaLN 6-modulation — 조건(t, OD)의 유일한 주입점; 이전 블록 내용은 여기로 흐르지 않음

보조 관찰: blk64(=MDLM 극한, 전 경로가 한 블록)에서는 OBC가 공집합이 되어 x_0 절반의 연산이 기능적으로 낭비가 되고(\$6.9의 각주), blk1(=AR 극한)에서는 모든 정보가 "커밋 블록 참조"로만 흘러 사실상 causal LMO이 된다 — block size가 이 두 통로(블록 내 BD vs 블록 간 OBC/BC)의 비율을 보간한다.